

Stop Generating the Code You Already Have: Chimera Shared Edit Programs for Faster Long-Context Code Editing

Ceyhun Derinbogaz

GPT-5.5

May 28, 2026

Abstract

Modern LLM serving stacks are optimized around the decode loop: generate the next token faster, schedule KV cache more efficiently, or speculate multiple tokens ahead. For coding agents, this leaves a large systems opportunity untouched. When the desired output is mostly a deterministic transformation of context already available to the client, the model should not be asked to regenerate the artifact token by token. We evaluate Chimera shared edit programs, a compact validated representation for long-context code edits. On 50 deterministic LongCodeBench-derived 100K-context file-edit episodes using Qwen/Qwen3.6-35B-A3B-FP8 served by vLLM 0.21.0 on a single H100, Chimera achieves a **1.78 $\times$ mean paired artifact-throughput speedup** over a minimal unified-diff baseline under normal vLLM, with 95% bootstrap CI [1.56 $\times$, 2.02 $\times$]. Under vLLM Qwen MTP speculative decoding, the same representation achieves **1.43 $\times$ mean paired speedup** over MTP unified diff, with 95% CI [1.26 $\times$, 1.61 $\times$]. All measured workloads render successfully for all 50 examples. The result is deliberately narrow: Chimera does not make the transformer emit raw tokens faster. It makes the system produce the final edited artifact faster by emitting fewer redundant tokens and rendering the rest deterministically.

1 The Decode Tax Nobody Talks About

LLM serving has become an arms race around the decode loop. vLLM introduced PagedAttention for efficient serving memory management [2]. Speculative decoding [3] and multi-token prediction [1] attempt to accept several future tokens per target-model step. These are real systems improvements.

But coding assistants often pay a tax before these optimizations matter: they ask the model to repeat information that already exists in the client. A file-edit assistant may receive a repository, identify a simple operation, and then emit thousands of tokens of file content or patch scaffolding whose semantics are mostly “copy the existing text and add this small change.” In that regime, the right question is not only how fast the model decodes tokens. It is why the model is decoding those tokens at all.

Chimera is a representation-level answer. The model emits a small validated edit program. The client applies it. The artifact appears without forcing the model to recite it.

2 Method

For copy-heavy code edits, the output artifact can be decomposed into large substrings already present in context, small novel edits, and a deterministic transformation that combines the two. Traditional full-file generation collapses all three into raw text generation. Unified diff improves the situation by emitting a patch. Chimera shared edit programs factor repeated operations across files.

The operation evaluated here is intentionally simple:

```
{
  "operation": "prepend_many",
  "paths": ["file_a.py", "file_b.py",
            "file_c.py", "file_d.py"],
  "text": "# CHIMERA_EDIT_MARKER: ...\n"
```

The client validates JSON parseability, operation type, exact path set, and exact inserted text. It then renders the final edited files from source artifacts already known to the harness. This is not a replacement for vLLM, MTP, or KV-cache optimization. It is a layer above them: engine-level acceleration asks the server to generate tokens faster; Chimera asks the model to generate fewer tokens that better describe the operation.

3 Experimental Setup

Hardware and runtime. Experiments run on a Nebius gpu-h100-sxm VM with 1 NVIDIA H100 80GB HBM3 GPU, driver 580.126.09, Qwen/Qwen3.6-35B-A3B-FP8 [5], and vLLM 0.21.0 [6]. The server uses max model length 131072, prefix caching, chunked prefill, MAX_NUM_SEQS=1, and MAX_NUM_BATCHED_TOKENS=8192. Decoding uses temperature 0.0 and top- p 1.0.

Workload. We build 50 deterministic file-edit episodes from LongCodeBench LongCodeQA [4]. The first user turn contains approximately 96K Qwen tokens of repository context. Each episode then asks the model to edit four target files by prepending the same marker line. The source rows are deterministically shuffled with seed 20260527. Every compared representation uses the same record IDs and target files.

Compared systems. We compare normal vLLM and vLLM Qwen MTP with two output representations: minimal unified diff and Chimera shared edit programs. We also include MTP depth 6 because an earlier chat-style sweep favored it; in this edit workload, depth 6 is slower than depth 2.

Table 1: Aggregate 50-example results. Every workload renders successfully for all 50 examples.

Profile	Workload	Render	Artifact tok/s	Output tok/s	Avg s
vLLM	diff	50/50	4586.19	65.47	2.62
vLLM	Chimera	50/50	7576.24	41.87	1.59
MTP-2	diff	50/50	4555.04	65.03	2.64
MTP-2	Chimera	50/50	5982.65	33.06	2.01
MTP-6	diff	50/50	4393.81	62.73	2.74
MTP-6	Chimera	50/50	5208.69	28.79	2.31

Metrics. model output tokens/s measures emitted model tokens per second. artifact tokens/s measures validated final edited-file tokens per wall-clock second. Chimera is designed to improve artifact tokens/s, not model output tokens/s.

4 Results

The strongest result is not that Chimera beats a weak full-file baseline. It beats a practical patch baseline. Against minimal unified diff, Chimera improves effective artifact throughput by $1.78\times$ on normal vLLM and by $1.43\times$ when MTP is enabled. The speedup survives both a real patch-format baseline and an engine-level speculative decoding control.

MTP is not a free win on this workload. Depth 2 slightly underperforms normal vLLM in aggregate artifact throughput, and depth 6 is slower than depth 2. This does not contradict earlier chat-trace measurements where MTP helped; it shows that speculative settings do not automatically transfer to copy-heavy edit-program workloads.

5 Discussion

Chimera wins here because the edit is shared across several files. Unified diff is already compact, but it repeats file-level patch structure. Chimera emits the operation once and points it at multiple paths. This distinction is small in syntax and large in systems behavior.

This is not KV caching. KV caching avoids recomputing past context; it does not stop the model from emitting redundant output tokens. This is not simply MTP either. MTP tries to traverse the token sequence faster. Chimera shortens the required sequence. The two ideas can be complementary, but this run shows they are not guaranteed to compose positively without workload-specific tuning.

6 Limitations

This is a systems result, not a universal model-quality result. It uses one model, one engine, one GPU class, one concurrency setting, and one deterministic edit family. The task intentionally isolates copy-render behavior; it does not measure arbitrary reasoning-heavy software engineering. The correct claim is that validated structured edit programs can improve artifact-level throughput for copy-heavy code edits. The incorrect claim

would be that the transformer itself decodes raw tokens faster.

7 Conclusion

The usual inference story says faster LLM systems need faster kernels, better KV-cache management, or speculative decoding. Those are real wins. This paper shows another path: stop generating the artifact when a validated program can render it.

On 50 LongCodeBench-derived 100K-context file-edit episodes, Chimera shared edit programs deliver a $1.78\times$ mean paired artifact-throughput speedup over minimal unified diff on normal vLLM and a $1.43\times$ speedup under MTP. For coding agents, the next major speedup may not come from making models type faster. It may come from teaching them when to stop typing.

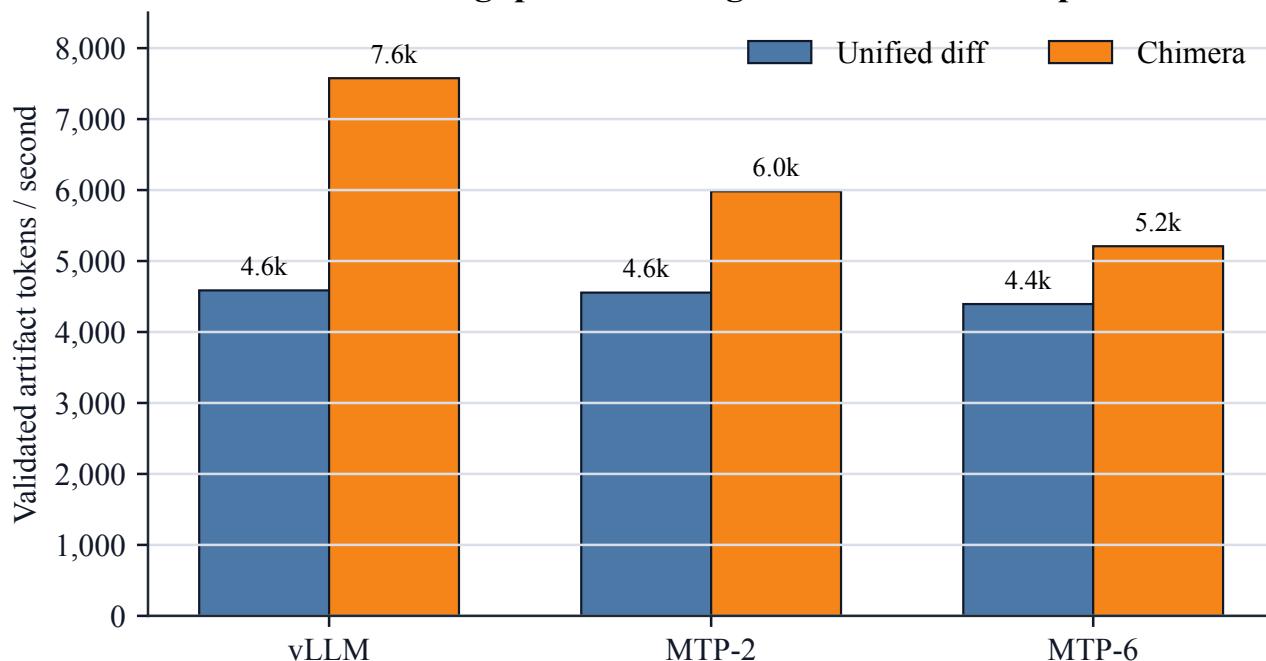
Reproducibility

The repository includes raw CSV summaries in `data/`, figure generation in `scripts/make_figures.py`, and a `Makefile`. The companion experiment repository contains the raw per-request logs, generated traces, and driver logs for the paper50 run.

References

- [1] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Roziere, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL <https://proceedings.mlr.press/v235/gloeckle24a.html>.
- [2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. *arXiv preprint arXiv:2309.06180*, 2023. URL <https://arxiv.org/abs/2309.06180>.
- [3] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, pages 19274–19286, 2023. URL <https://proceedings.mlr.press/v202/leviathan23a.html>.
- [4] LongCodeBench Authors. Longcodebench: Evaluating coding llms at 1m context windows. *arXiv preprint arXiv:2505.07897*, 2025. URL <https://arxiv.org/abs/2505.07897>.
- [5] Qwen Team. Qwen/Qwen3.6-35B-A3B-FP8. <https://huggingface.co/Qwen/Qwen3.6-35B-A3B-FP8>, 2026.

Artifact throughput on 50 long-context code-edit episodes



Qwen3.6-35B-A3B-FP8, vLLM 0.21.0, 1x H100, 100K-context LongCodeBench-derived edits.

Figure 1: Validated artifact throughput. Chimera shared edit programs produce the same validated edited artifacts faster than minimal unified diff under both normal vLLM and MTP.

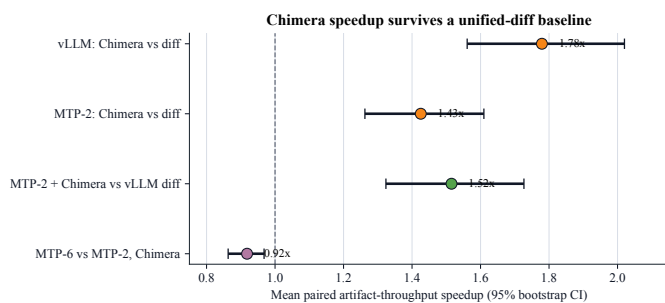


Figure 2: Paired per-example speedups. Error bars show 95% bootstrap confidence intervals over matching record IDs.

[6] vLLM Project. vLLM: Easy, fast, and cheap llm serving. <https://github.com/vllm-project/vllm>, 2026.

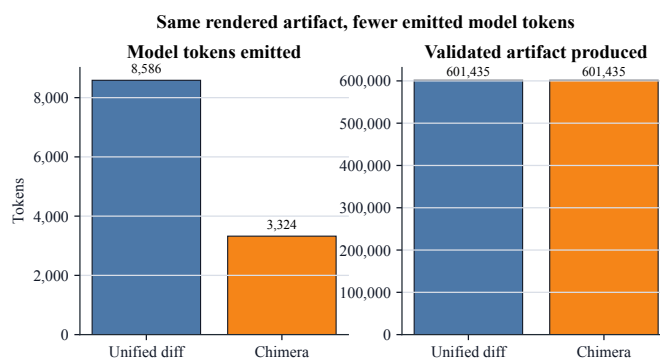


Figure 3: Same artifact, fewer emitted tokens. Chimera improves artifact throughput by reducing redundant output generation; it does not claim faster raw token emission.

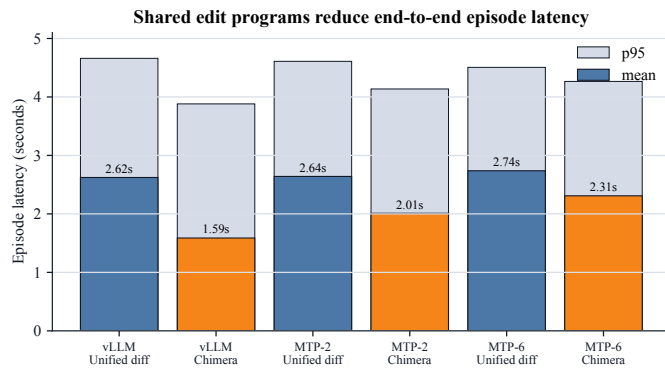


Figure 4: Episode latency. Bars show mean latency; gray background bars show p95.